Design Space Exploration and Layer-Aware Mapping for Systolic Array Variants

Haocheng Fan, Yaohan Huang, Yixu Pan, Bokang Wu
Github URL: https://github.com/frankfan114/ADLS_Group_11.git

Abstract

This report presents a hardware-oriented framework for mapping quantized neural network(NN) layers to systolic-array implementations. The project focuses on a common compute building block underlying multiple NN operators. Multiple variants—including output-stationary (OS), weight-stationary (WS), buffered (ping-pong), and a flexible reconfigurable WS-based architecture are designed and evaluated, under a unified benchmarking flow to analyze trade-offs in latency, throughput, utilization, stall behavior, and hardware cost.

Building on this design space, a lightweight automated toolflow that ranks candidate configurations using benchmark data and generates the corresponding RTL is developed. The flow combines offline profiling with online selection, enabling layer-aware mapping without exhaustive evaluation.

Results demonstrate clear trade-offs across dataflow, buffering, and flexibility. Buffering is beneficial only with sufficient overlap, while reconfiguration improves utilization under shape mismatch but does not eliminate memory bottlenecks. Overall, the work highlights both architectural trade-offs and the effectiveness of benchmark-driven automated mapping.

I. Introduction

This project has two objectives: (1) to design and evaluate multiple systolic-array architectures for representative quantized neural network workloads, and (2) to develop an automated tool that selects an appropriate design based on layer characteristics and user preferences. Rather than covering diverse operators, the focus is on a shared compute primitive common to matrix multiplication and convolution, examined through multiple hardware realizations under a unified flow. Since inference is dominated by regular compute patterns, performance is primarily governed by dataflow, reuse, memory behavior, and array utilization, making architectural choices central to performance–resource trade-offs.

Systolic arrays are well suited to these workloads due to their regular structure and scalable parallelism. However, no single design is optimal across all layer shapes. Different workloads favor different strategies, including partial-sum locality, weight reuse, communication–computation overlap, and adaptability to varying dimensions, motivating the exploration of multiple variants rather than a single baseline.

This work implements and compares several designs within a common framework: an output-stationary (OS) baseline, a weight-stationary (WS) baseline, buffered variants for improved overlap, and a reconfigurable design derived from WS [1]. These variants expose distinct trade-offs in latency, throughput, utilization, stall behavior, and resource cost.

Building on this design space, a lightweight automated toolflow is developed to support layer-aware mapping. The selector uses benchmark data from the implemented architectures to rank candidate configurations and generate corresponding RTL setups for evaluation, enabling a reproducible and practical mapping workflow.

II. Overall Architecture

Figure 1 illustrates the proposed system architecture. The toolflow maps a quantized neural network layer to an appropriate implementation within a predefined array size configurable systolic-array design space using benchmark-driven profiling, automated ranking, and configuration generation.

The flow consists of two stages: an **offline profiling stage** and an **online selection-and-generation stage**.

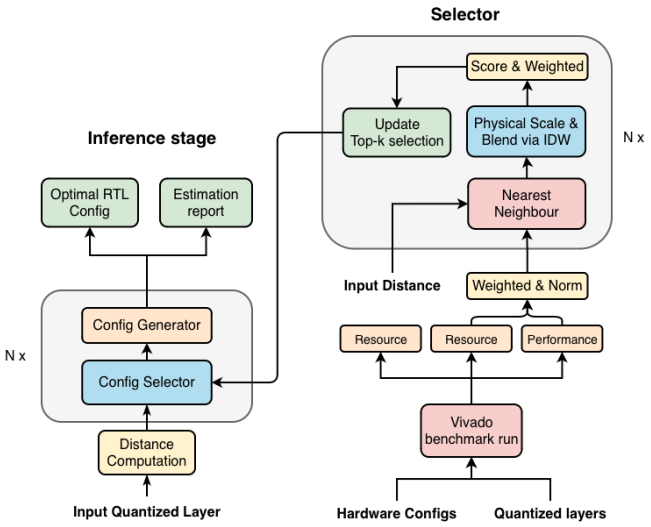

Fig. 1: Overall system architecture.

In the offline stage, representative quantized workloads are evaluated across multiple systolic-array variants under a unified benchmark setup. Metrics such as latency, throughput, PE utilization, stall ratio, off-chip traffic, and resource proxies are collected to form a reference database capturing architectural trade-offs.

In the online stage, layer-level inputs (e.g., matrix dimensions and bit-width) are used to estimate candidate performance by comparison with the reference database. Configurations are ranked via a weighted scoring function, with

interpolation enabling generalization to unseen layers. The top-ranked (or top-k) configurations are selected.

Finally, the system instantiates the chosen design by selecting the architectural variant, assigning parameters, and generating the corresponding RTL configuration. This enables both systematic design-space exploration and practical hardware realization for new workloads.

III. Hardware Design and Trade-offs

Several systolic-array-based hardware variants are designed and evaluated for quantized neural network workloads. They are organized as a unified hardware design space with different optimisation priorities: accumulation locality, weight reuse, communication–computation overlap, and workload adaptability. This section describes the architectural role of each variant and the concrete implementation decisions required to realize and compare them under consistent conditions.

The core design space includes four main selectable variants: an output-stationary (OS) baseline, a weight-stationary (WS) baseline, ping-pong-buffer-enhanced buffered variants, and a flexible reconfigurable variant derived from the WS design. In addition, a sparsity-oriented extension is implemented as an exploratory design, although it is not included in the main selector flow, as FPGA-deployed models are typically compact and resource-constrained, leaving limited redundancy to leverage.

A. Output-Stationary Baseline

The OS design serves as the simplest dense baseline. It follows the standard output-stationary dataflow [2]: each PE accumulates partial sums locally, activations and weights propagate through the array in a regular wavefront, and outputs are written back only after the full reduction along the K dimension is completed. From an implementation perspective, this results in the cleanest controller structure among all variants. The control path mainly consists of tiled operand feeding, wavefront timing, output-tile completion detection, and final draining of accumulated results.

A deliberate design decision in the OS baseline is that weight tiles are not preserved across multiple output-row sweeps. Once a tile has been consumed for the current schedule, later reuse is handled by refetch rather than by explicit on-chip residency. This keeps the architecture and controller easy to reason about and makes OS a clean reference point for later comparison. The downside is predictable: for workloads with significant cross-tile weight reuse, off-chip traffic can become unnecessarily high. This behaviour is intentionally retained, since introducing weight-residency logic would blur the distinction between OS and WS and reduce baseline interpretability.

In summary, OS is not intended to represent the most aggressive optimisation point in the design space. Instead, it provides a control-minimal dense systolic baseline with strong partial-sum locality and a regular execution pattern.

B. Weight-Stationary Baseline

The WS design targets a different bottleneck: repeated weight movement [3], [4]. In this architecture, weight tiles are kept resident inside the array or in tightly coupled local storage, while activation tiles stream through the datapath. This modifies both the memory schedule and controller requirements relative to OS. Instead of treating weights as transient operands, the controller explicitly manages weight loading, validity, reuse windows, and eventual tile replacement.

The key implementation choice concerns loop ordering and residency policy. The controller is structured such that a weight tile can remain active across multiple output-row sweeps whenever reuse is available, while activations advance according to the current tile schedule. This requires additional state compared with OS, including tile-valid flags, reuse-aware progression conditions, and sequencing logic for tile replacement. These mechanisms are essential for enabling sustained weight reuse. Otherwise, the design degenerates into a streamed architecture with added control overhead but limited traffic reduction.

This design is well suited to inference-like workloads with predictable weight reuse. However, the benefit comes at the cost of increased control complexity, less regular scheduling, and tighter coupling between compute progression and memory-side tile management. These trade-offs directly reflect the primary objective of WS: reducing repeated weight fetches in reuse-heavy workloads.

C. Buffered Variants

To mitigate stalls caused by memory-side latency, the dense baselines are extended with ping-pong-buffer-based buffered variants. Buffering is introduced around the data supply and writeback path rather than within the systolic compute core. As a result, the MAC datapath, PE timing, and array wavefront remain unchanged, while memory access and bank switching are reorganized to enable overlap between communication and computation.

In practice, two buffer banks are employed with alternating roles. One bank serves the active compute phase, while the other is refilled or drained for subsequent phases. This enables partial hiding of memory latency behind ongoing execution. Since the compute core remains unchanged, performance differences can be attributed to improved data staging rather than modifications to the arithmetic pipeline.

The buffered OS and buffered WS variants exhibit different behaviours. In OS+PP, the relatively regular compute schedule allows overlap to be exposed more effectively. The use of split read and write paths further reduces contention and stabilizes latency improvements. In WS+PP, however, buffering interacts with weight residency and reuse-dependent scheduling. This complicates arbitration, as reuse timing, read servicing, and output draining must be coordinated carefully. As a result, gains may be smaller, or even negative, when overlap opportunities do not sufficiently compensate for the additional control and bank-management overhead.

Buffering is therefore treated as a conditional optimisation. It is beneficial only when memory transfer can be meaningfully decoupled from computation. Otherwise, the added state, arbitration logic, and coordination cost may outweigh its advantages.

D. Flexible Reconfigurable Variant

The flexible variant is built on top of the WS architecture [5]. This choice reflects the fact that WS already relies on explicit scheduling state and tile residency management, providing a natural foundation for introducing array-shape adaptation. Extending OS in a similar manner would require substantial restructuring of its simpler control flow.

The motivation for this variant is array underutilization under mismatched workload shapes. A fixed $m \times n$ array may perform efficiently when the workload aligns with its aspect ratio, but can suffer from low utilisation for skewed or narrow layer shapes. To address this, the design supports a bounded set of alternative active array configurations. Arbitrary resizing is avoided in favour of a discrete configuration set to maintain manageable verification and control complexity.

Flexibility is implemented through configuration state, active-lane masking, and logical remapping between workload tiles and physical PE regions. During execution, only selected rows and columns are activated, while the controller adjusts scheduling and tile interpretation accordingly. This improves effective utilisation across heterogeneous workloads.

The associated cost is increased hardware and control complexity, including additional configuration handling, enable logic, mapping support, and less regular memory access patterns. Consequently, this variant is not expected to outperform fixed designs on well-matched workloads. Its role is to improve average efficiency across diverse layer shapes.

E. Sparse Extension

A sparsity-oriented extension is also implemented to reduce ineffective computation when activations or weights contain many zero-valued entries. Instead of adopting compressed formats or global sparse scheduling, a local approach is used: an SMT-like zero-skipping mechanism at the PE level [6].

This approach maintains compatibility with dense tile formats, the existing memory interface, and the shared benchmarking infrastructure. It isolates compute-side sparsity effects without introducing a separate sparse software stack. However, it does not reduce off-chip traffic, as operands remain stored and transferred in dense form.

This variant is treated as exploratory and is not included in the main selection flow. The evaluated workloads are relatively small and do not exhibit significant redundancy, limiting the observable benefits of sparsity.

IV. Performance Metrics

Evaluation targets hardware efficiency and design trade-offs, while accuracy is used only to validate workloads. All results are obtained under a DDR-backed memory model (Xilinx MIG 7-series).

- **Latency:** End-to-end execution time (cycles).
- **Throughput:** Sustained work per cycle (e.g., ops/cycle).
- **PE Utilization:** Fraction of cycles with active computation.
- **Stall Ratio:** Fraction of cycles stalled by data/memory limits.
- **Resource Usage:** LUTs, FFs, and MUXes.

Latency and throughput capture performance, utilization and stalls indicate bottlenecks, and resource usage reflects implementation cost.

V. Testing

A. Functional Verification

Each hardware variant was verified using cocotb-based tests, covering arithmetic correctness, control logic, and data movement. Both directed corner cases and random regression tests were used to exercise boundary conditions, tile transitions, and representative inputs. Detail cases are shown in code.

For selected workloads, outputs were compared against software references to validate quantized execution, accumulation behaviour, and correctness across different dataflows.

B. Performance Benchmarking

All systolic-array variants were benchmarked using a shared Vivado-based flow under a DDR-backed memory model, ensuring cycle counts and stalls reflect off-chip effects.

Latency, throughput, utilization, stall ratio, and resource metrics are collected. These results enable architectural comparison and populate the selector's reference database, making benchmarking an integral part of the toolflow.

VI. Performance Analysis

A. 4×4 variants

The Xilinx Artix-7 XC7A200TSBV484-1 FPGA is used as the target deployment platform. Designs are evaluated across three PE array sizes (4×4, 8×8, and 16×16), while the detailed analysis below focuses on the 4×4 configuration.

Design	Lat.	Thr.	Util.	Stall
OS Baseline	76 033	0.108	0.17%	72.2%
WS Baseline	43 862	0.187	0.29%	69.8%
OS + Buffered	75 675	0.108	0.17%	72.6%
WS + Buffered	44 506	0.184	0.29%	70.2%
Reconfigurable WS	44 099	0.186	1.16%	70.0%

TABLE I: Hardware variant summary on BERT-Tiny layer (M=32, K=128, N=2, 4×4 PE array).

Table I shows that the primary distinction on this BERT-Tiny layer is between OS- and WS-style execution rather than buffering or reconfiguration. The WS baseline clearly outperforms OS, reducing latency from 76,033 to 43,862 cycles (42.3%) and increasing throughput from 0.108 to 0.187 (73.1%), which indicates that this workload benefits more from explicit weight residency than from OS-style partial-sum locality. In contrast, buffering brings little benefit: OS+Buffered is almost unchanged relative to OS, while WS+Buffered is

slightly worse than WS in both latency and throughput, suggesting that the available communication–computation overlap is too limited to amortize the additional bank-management and arbitration overhead. The reconfigurable WS variant achieves performance close to the fixed WS baseline while raising PE utilization substantially from 0.29% to 1.16%, indicating improved mapping efficiency under this layer shape. However, because the stall ratio remains high at 70.0%, this utilization gain does not translate into a comparable throughput improvement. Overall, the results suggest that this layer is dominated by memory/data-delivery effects, with WS providing the best fixed design point, buffering offering negligible benefit, and reconfiguration primarily improving array matching rather than removing the main bottleneck.

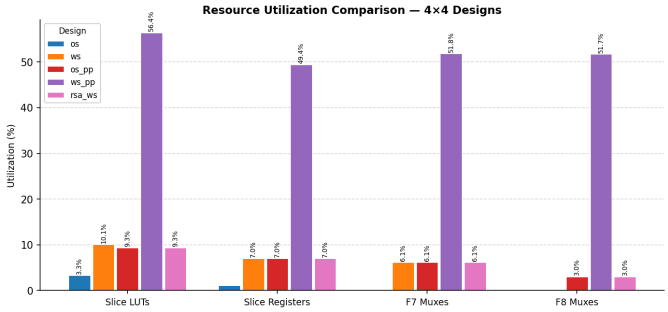

Fig. 2: Resource utilization for 4×4 configuration.

Resource utilisation from Graph 2 shows a much sharper separation between variants than the runtime table. OS is the clear area-minimal baseline at 4,451 LUTs and 3,050 registers, with negligible mux overhead, confirming that its simple streaming schedule and lack of explicit operand residency keep control and routing costs low. Moving to WS increases cost substantially to 13,545 LUTs and 18,863 registers, reflecting the additional state and steering logic required to hold and reuse weights across cycles. This is the expected price of converting weight reuse into on-chip structure. OS+Buffered occupies a similar middle tier, with 12,578 LUTs and 18,891 registers, indicating that the ping-pong staging infrastructure costs roughly as much area as the WS residency machinery at this array size. By contrast, WS+Buffered is clearly impractical on this device, consuming 75,932 LUTs and 132,951 registers, together with more than half of available F7/F8 mux resources. This suggests that duplicating and arbitrating buffered weight-resident state is the dominant area blow-up rather than the MAC datapath itself. Reconfigurable WS has the same reported footprint as OS+Buffered, which implies that, at 4×4 scale, the added flexibility is effectively area-neutral relative to moderate buffering overhead. Finally, DSP usage is constant across all designs, so the architectural trade-off is governed almost entirely by LUT, register, and mux growth rather than arithmetic resources. Overall, the resource results place OS as the most deployment-friendly design under tight area budgets, WS and reconfigurable/buffered moderate-cost designs.

B. Selector

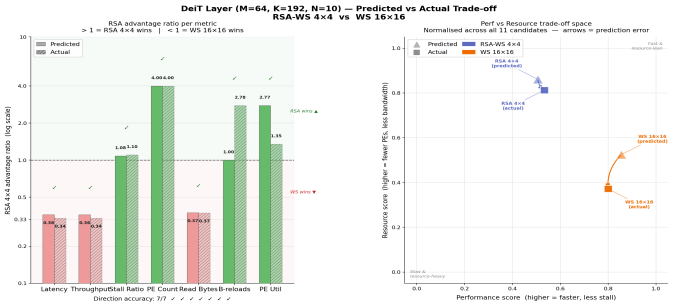

Fig. 3: Selector produced suggest design trade-off for the DeiT layer (M=64, K=192, N=10) with RSA-WS 4×4 and WS 16×16.

The selector is built from benchmark data collected on two representative workloads spanning different model families and quantization regimes: (1) the MNIST MLP classification head, and (2) the BERT-Tiny output projection head. Together, they capture key hardware behaviours—moderate-precision accumulation and low-precision projection—allowing generalization to similar fully connected layers. Larger workloads were not evaluated due to resource constraints.

DeiT linear layer (M=64, K=192, N=10) outside the reference dataset is tested to validate the selector. The selector ranks **WS 16×16** first and **RSA-WS 4×4** second based on predicted performance–resource trade-offs. Both configurations are implemented and measured on hardware. Figure 3 shows that the selector correctly predicts the direction of all seven metrics: **WS 16×16** achieves lower latency and memory traffic (predicted 153,414 vs. actual 127,520 cycles; 14,592 read bytes), while **RSA-WS 4×4** uses fewer PEs (16 vs. 64) and has a lower stall ratio, making it more resource-efficient despite higher latency. The normalized performance–resource plot further shows close agreement between predicted and measured positions, indicating low prediction error.

Overall, the selector accurately captures the trade-off between compute-optimized and resource-efficient designs.

VII. Conclusion

This work studies systolic-array variants for quantized neural networks and introduces a lightweight flow for layer-aware mapping. The design space spans OS, WS, buffered, and flexible architectures, each trading off locality, reuse, overlap, and adaptability, leading to measurable differences in latency, throughput, utilization, stalls, and resource cost.

The proposed selector shows that benchmark-driven profiling can enable practical layer-aware mapping and automated RTL generation for unseen workloads. While evaluated on a limited set, the framework is reproducible and extensible to broader design spaces and mapping strategies.

REFERENCES

[1] R. Xu, S. Ma, Y. Guo, and D. Li, "A survey of design and optimization for systolic array-based dnn accelerators," *ACM Computing Surveys*, vol. 56, no. 1, pp. 1–37, 2023.

[2] H.-T. Kung *et al.*, "Why systolic architecture?" 1982.

[3] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *Proceedings of the 44th annual international symposium on computer architecture*, 2017, pp. 1–12.

[4] H. Genc, A. Haj-Ali, V. Iyer, A. Amid, H. Mao, J. Wright, C. Schmidt, J. Zhao, A. Ou, M. Banister *et al.*, "Gemmini: An agile systolic array generator enabling systematic evaluations of deep-learning architectures," *arXiv preprint arXiv:1911.09925*, vol. 3, no. 25, pp. 15–17, 2019.

[5] A. Samajdar, E. Qin, M. Pellauer, and T. Krishna, "Self adaptive reconfigurable arrays (sara) learning flexible gemm accelerator configuration and mapping-space using ml," in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, 2022, pp. 583–588.

[6] G. Shomron, T. Horowitz, and U. Weiser, "Smt-sa: Simultaneous multithreading in systolic arrays," *IEEE Computer Architecture Letters*, vol. 18, no. 2, pp. 99–102, 2019.